

Indian Institute of Technology Guwahati

CS525 Project Report

LLM-Guided Decision Heuristics for SAT-Based Automatic Test Pattern Generation

Progress Report

Course CS525
Date March 25, 2026

Team Members

Name	Roll Number
Shrey Chandra	230101097
Jagrati Mittal	230101047
Divyansh Malviya	230101037
Harsh	230101045

Contents

1	Introduction	1
2	Project Objectives	1
3	Work Completed So Far	2
3.1	Baseline SAT-ATPG Engine	2
3.2	Netlist Parsing Support	2
3.3	Benchmark Evaluation Flow	2
3.4	LLM-Guided ATPG Comparison Flow	3
4	Methodology	3
4.1	SAT Formulation	3
4.2	Fault Model	4
4.3	Baseline Metrics	4
4.4	Guided SAT Strategy	4
5	Implementation Details	5
5.1	Main Source Files	5
5.2	Important Fixes Made During Development	5
6	Preliminary Baseline Results	5
6.1	Baseline SAT-ATPG on Selected Benchmarks	5
7	Preliminary LLM-Guided Results	6
7.1	Selective Guidance Policy	6
7.2	Guided Comparison Results	6
8	Current Limitations	7
9	Future Work	7
10	Conclusion	8

Abstract

This report presents the implementation progress of a SAT-based Automatic Test Pattern Generation (ATPG) framework for combinational circuits, along with an initial LLM-guided extension. The implemented system supports ISCAS-style gate-level Verilog netlists and Yosys JSON netlists, converts circuits into CNF, injects stuck-at faults, and uses MiniSat22 through PySAT to generate tests and collect solver metrics. A baseline benchmark evaluation flow has been implemented, and an LLM-guided comparison framework has been added using assumption-based guidance with fallback. Preliminary results show that a selective guidance strategy can improve runtime and solver statistics on several benchmarks while preserving fault coverage.

1 Introduction

Automatic Test Pattern Generation (ATPG) is a fundamental problem in digital testing. In SAT-based ATPG, the goal is to determine whether there exists a primary input assignment that distinguishes a fault-free circuit from a faulty circuit under a given structural fault model. The assignment for this project specifically asks for:

1. a SAT-based ATPG core,
2. support for benchmark evaluation,
3. an LLM-guided heuristic layer, and
4. solver-side measurements such as decisions, conflicts, and runtime.

The current work implements the core SAT-ATPG engine and an initial guided comparison framework. The system is designed to operate on both benchmark gate-level netlists and technology-mapped netlists exported by Yosys in JSON form.

2 Project Objectives

The main objectives of the project are:

1. Build a SAT-based ATPG framework for stuck-at faults.
2. Support both ISCAS gate-level Verilog and Yosys JSON netlists.
3. Evaluate fault coverage and SAT-level metrics on benchmark circuits.
4. Integrate an LLM layer that provides useful partial assignments or guidance.
5. Compare baseline SAT-ATPG against guided SAT-ATPG using measurable metrics.

3 Work Completed So Far

3.1 Baseline SAT-ATPG Engine

The following core components have been implemented:

1. **Gate-to-CNF encoder:** CNF encodings have been implemented for AND, OR, NOT, BUF, NAND, NOR, and XOR.
2. **Good/Faulty circuit construction:** For each fault, separate good and faulty copies of the circuit are built.
3. **Fault injection:** Stuck-at-0 and stuck-at-1 faults are supported.
4. **Fault-site handling:** In the faulty copy, the original driver of the fault site is disconnected and the stuck-at value is imposed.
5. **Primary input consistency:** Good and faulty primary inputs are tied together, except when the fault is itself on a primary input.
6. **Output miter:** A multi-output miter is implemented so that the SAT instance is satisfiable if any primary output differs between the good and faulty circuits.
7. **SAT solving:** PySAT with Minisat22 is used for solving.

3.2 Netlist Parsing Support

Two parser flows are currently supported:

1. **ISCAS-style Verilog parser:** Handles benchmark-style gate-level Verilog and supports multiline input/output declarations.
2. **Yosys JSON parser:** Parses Yosys-generated JSON netlists and normalizes supported cell types into the internal gate representation.

This satisfies the requirement that the ATPG system should work on both benchmark netlists and technology-mapped netlists.

3.3 Benchmark Evaluation Flow

A benchmark runner has been implemented to:

1. generate all single stuck-at faults,
2. run ATPG over all faults,
3. measure detection count,
4. collect SAT time and wall-clock time,

5. collect MiniSat statistics such as decisions and conflicts, and
6. export the results to CSV.

The runner also supports:

1. **quick mode** for faster development,
2. **selected benchmark mode**, and
3. configurable CSV output.

3.4 LLM-Guided ATPG Comparison Flow

A separate comparison module has been implemented to avoid disturbing the baseline system. The guided flow currently works as follows:

1. baseline solve is run first for each fault,
2. hard faults are identified using solver thresholds,
3. a guidance module proposes partial primary input assignments,
4. these assignments are passed as SAT assumptions,
5. if the guided solve fails, the flow falls back to normal SAT solving,
6. the framework reports baseline vs guided metrics.

The current implementation supports:

1. heuristic guidance,
2. OpenAI-backed guidance,
3. Gemini-backed guidance,
4. selective invocation on hard faults only,
5. fallback counting, and
6. UNSAT core size logging for guided assumptions.

4 Methodology

4.1 SAT Formulation

For a given target fault f , the ATPG problem is encoded into CNF using:

1. a CNF encoding of the *good* circuit,

2. a CNF encoding of the *faulty* circuit,
3. a fault constraint enforcing the stuck-at condition at the fault site, and
4. a miter that enforces output divergence.

A SAT solution corresponds to a valid test vector.

4.2 Fault Model

The current implementation supports the single stuck-at fault model:

- stuck-at-0,
- stuck-at-1.

For each signal s in the circuit, two faults are considered:

$$(s, sa0), \quad (s, sa1)$$

4.3 Baseline Metrics

For each benchmark, the following metrics are collected:

- detected faults,
- total faults,
- fault coverage,
- SAT solving time,
- wall-clock runtime,
- average decisions per fault,
- average conflicts per fault.

4.4 Guided SAT Strategy

The guided ATPG flow currently uses *assumption-based guidance*. The model is asked to suggest a small set of primary input assignments. These are applied as SAT assumptions. To avoid harming easy faults:

1. guidance is applied only to hard faults,
2. the number of assumptions is capped,
3. if assumptions make the instance fail, the solver falls back to standard SAT.

5 Implementation Details

5.1 Main Source Files

The project currently contains the following important modules:

- `src/cnf_encoder.py`: primitive CNF encodings and miter construction.
- `src/sat_atpg_demo.py`: baseline ATPG engine, parser logic, fault CNF construction.
- `src/benchmark_runner.py`: baseline benchmark evaluation and CSV export.
- `src/sat_atpg_llm_demo.py`: baseline-vs-guided comparison framework.
- `src/toy_demo.py`: small sanity-check ATPG example.

5.2 Important Fixes Made During Development

Several correctness issues were identified and fixed during development:

1. incorrect NAND CNF encoding,
2. incorrect multi-output miter encoding,
3. failure to disconnect the fault site from its original driver in the faulty copy,
4. incorrect handling of multiline input/output declarations,
5. first-output-only checking replaced by proper multi-output divergence.

These fixes significantly improved measured fault coverage on benchmark circuits.

6 Preliminary Baseline Results

6.1 Baseline SAT-ATPG on Selected Benchmarks

Table 1 shows the currently validated baseline metrics on the faster benchmark subset.

Table 1: Baseline SAT-ATPG Results on Selected Benchmarks

Benchmark	Faults Evaluated	Detected / Total	Coverage (%)	Wall Time (s)	SAT Time (s)
c432	392	389 / 392	99.23	0.529	0.248
c499	486	486 / 486	100.00	2.145	1.664
c880	886	886 / 886	100.00	3.060	1.640
c1355	1174	1174 / 1174	100.00	6.979	4.419
s298	284	284 / 284	100.00	0.151	0.006
s344	380	380 / 380	100.00	0.242	0.009

The larger circuits **c3540** and **c6288** are considerably more expensive, so limited-fault evaluation is used during guided comparison experiments.

7 Preliminary LLM-Guided Results

7.1 Selective Guidance Policy

A selective policy was adopted because applying guidance on every fault often hurt performance. The current strategy is:

1. run baseline per fault,
2. classify a fault as hard if:

$$\text{conflicts} \geq 100 \quad \text{or} \quad \text{decisions} \geq 800$$

3. apply guidance only on hard faults,
4. use a maximum of 4 assumptions,
5. reduce assumptions for very hard faults.

7.2 Guided Comparison Results

Table 2 summarizes preliminary baseline-vs-guided comparison results using the current selective heuristic-guidance strategy.

Table 2: Preliminary Baseline vs Guided Comparison

Benchmark	Faults Evaluated	Coverage Preserved	Baseline SAT (s)	Guided SAT (s)	Baseline Conflicts	Guided Conflicts
c432	392	Yes	0.235	0.179	125.84	90.72
c499	486	Yes	1.585	1.098	710.81	519.65
c880	500	Yes	0.802	0.939	226.07	270.70
c1355	500	Yes	2.646	2.453	625.80	593.48
s298	284	Yes	0.006	0.006	1.52	1.52
s344	380	Yes	0.009	0.009	1.84	1.84

These results indicate:

1. guidance can help on some benchmarks,
2. selective gating prevents regressions on easy circuits,
3. the quality of guidance still depends strongly on the hint generator.

8 Current Limitations

The current work is still preliminary and has several limitations:

1. The guided model currently uses simple assumption-based hints rather than richer structural reasoning.
2. OpenAI and Gemini integration depends on external API access, quota, and response reliability.

Free Tier Limits (Approximate)

Since you aren't using a billing address, your usage is capped to prevent abuse. While these numbers can shift, the typical limits are:

Model	Requests Per Minute (RPM)	Requests Per Day (RPD)
Gemini 2.5 / 3 Flash	10–15	~250–1,000
Gemini 2.5 Pro	2–5	~50–100

Figure 1: Illustration of API quota limits affecting LLM-guided ATPG runs

3. Benchmark-heavy circuits such as `c3540` and `c6288` are time-consuming in full-fault evaluation.
4. Statistical significance testing has not yet been added.
5. The current implementation is restricted to combinational stuck-at fault testing.

9 Future Work

The next steps for the project are:

1. improve prompt construction using local fanin/fanout cones,
2. use per-fault structural summaries instead of generic PI hints,
3. export richer per-fault CSV logs,
4. compare heuristic guidance against real LLM guidance once API access is stable,
5. extend evaluation to technology-mapped JSON netlists in a systematic way,
6. investigate transition-fault support if time permits.

10 Conclusion

A functional SAT-based ATPG framework has been implemented with support for both benchmark Verilog netlists and Yosys JSON netlists. The system generates test patterns for stuck-at faults, collects solver metrics, and supports large-scale benchmark evaluation. A separate LLM-guided ATPG framework has also been developed, including selective assumption-based guidance and fallback-safe comparison against baseline SAT solving. Preliminary results show that guided ATPG can improve performance on several benchmarks, though more work is needed to make the guidance consistently beneficial across all circuits.

Reproducibility Commands

The following commands are used to reproduce the current work.

Baseline Commands

```
python src/toy_demo.py
```

```
python src/sat_atpg_demo.py
```

```
python -u src/benchmark_runner.py --quick
```

```
python -u src/benchmark_runner.py
```

```
python -u src/benchmark_runner.py --bench c432.v --bench c880.v
```

Guided Comparison Commands

```
python src/sat_atpg_llm_demo.py --bench c432.v --llm-mode heuristic \
  --fault-limit 200 --hard-conflicts 100 --hard-decisions 800 \
  --max-assumptions 4
```

```
python src/sat_atpg_llm_demo.py --suite --llm-mode heuristic \
  --big-limit 1200 --hard-conflicts 100 --hard-decisions 800 \
  --max-assumptions 4 --csv llm_compare_metrics_final.csv
```

Optional API-Based Runs

```
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-4o-mini"
```

```
python src/sat_atpg_llm_demo.py --bench c432.v --llm-mode openai \
  --fault-limit 100 --openai-debug
```

```
export GEMINI_API_KEY="..."
export GEMINI_MODEL="gemini-1.5-pro"

python src/sat_atpg_llm_demo.py --bench c432.v --llm-mode gemini \
  --fault-limit 100 --openai-debug
```